Experiment No. – 3							
Date of Performance:	**31/07/26**						
Date of Submission:	**31/07/26**						
Conceptual Understanding (02)	**Execution (1.5)**	**Problem Solving & Debugging(02)**	**Professional Conduct & Lab Ethics(02)**	**Output/ Result Accuracy (1.5)**	**Documentation & Result Analysis (01)**	**Experiment Total (10)**	**Sign with Date**

Experiment No. 3
Study of Constructors

3.1 Aim: Study of Constructors: Create a class 'Student'; with three data members which are name, age and address. The constructor of the class assigns default values name as "unknown";, age as '0'; and address a "not available". It has two members with the same name 'setInfo'. First method has two parameters for name and age and assigns the same whereas the second method has three parameters which are assigned to name, age and address respectively. Print the name, age and address of 10 students. Hint - Use an array of objects.

3.2 Course Outcome: Apply fundamental Java programming concepts, including data types, operators, control structures, and functions, to develop basic programs.

3.3 Learning Objectives: Constructor and its application in java programming.

3.4 Requirement: Java development kit (JDK as per OS)

3.5 Related Theory:

In Java, **a constructor** is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling the constructor, memory for the object is allocated in the memory.It is a special type of method which is used to initialize the object. Every time an object is created using the new() keyword, at least one constructor is called.It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

Types of constructors in JAVA:
There are two types of constructors in Java:
1 Default Constructor
2. Parameterized constructor

It is called a constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. It is because the Java compiler creates a default constructor if your class doesn't have any.

Rules for creating Java constructor

There are two rules defined for the constructor.

1.Constructor name must be the same as its class name

2. A Constructor must have no explicit return type

3. A Java constructor cannot be abstract, static, final, and synchronized

Java Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameters.

Syntax of default constructor:

<class_name>()

{ }

Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

```
//Java Program to create and call a default constructor
class Bike1{
 Bike1(){
System.out.println("Bike is created");}
public static void main(String args[]){
//calling a default constructor
```

```
Bike1 b=new Bike1();
}
}
```

Example of default constructor that displays the default values

```
Class  Student3{
int id;
String name;
//method to display the value of id and name
void display()
{System.out.println(id+" "+name);}
 public static void main(String args[]){
//creating objects
Student3 s1=new Student3();
Student3 s2=new Student3();
//displaying values of the bject
s1.display();
s2.display();
}
}
```

Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.
The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor

In this example, we have created the constructor of the Student class that has two parameters. We can have any number of parameters in the constructor.

```
//Java Program to demonstrate the use of the parameterized constructor.
class Student4{
int id;
String name;
//creating a parameterized constructor
Student4(int i,String n){
id = i;
name = n;       }
//method to display the values
void display()
{
System.out.println(id+" "+name);
}
public static void main(String args[]){
//creating objects and passing values
Student4 s1 = new Student4(111,"Karan");
Student4 s2 = new Student4(222,"Aryan");
//calling method to display the values of object
s1.display();
s2.display();
}
}
```

Constructor Overloading in Java

In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods. Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

Example of Constructor Overloading

```
class Box {
        double width, height, depth;
        // constructor used when all dimensions specified
        Box(double w, double h, double d)
```

```
        {
                width = w;
                height = h;
                depth = d;
        }
        // constructor used when no dimensions specified
        Box()
{ width = height = depth = 0; }

        // constructor used when cube is created
        Box(double len) { width = height = depth = len; }
        // compute and return volume
        double volume() { return width * height * depth; }
}
// Driver code
public class Test {
        public static void main(String args[])
        {
                // create boxes using the various constructors
                Box mybox1 = new Box(10, 20, 15);
                Box mybox2 = new Box();
                Box mycube = new Box(7);
                double vol;
                // get volume of first box
                vol = mybox1.volume();
                System.out.println("Volume of mybox1 is " + vol);
                // get volume of second box
                vol = mybox2.volume();
                System.out.println("Volume of mybox2 is " + vol);

                // get volume of cube
```

```
                vol = mycube.volume();
                System.out.println("Volume of mycube is " + vol);
        }
    }
```

Array of objects: Array: An array is a collection of similar data elements stored at contiguous memory locations. It is the simplest data structure where each data element can be accessed directly by only using its index number.It is used with objects to create an array of objects.

For Eg:

For creating an array of objects of class vehicle.

vehicle[] obj = new vehicle[size of array];

3.6 Procedure:

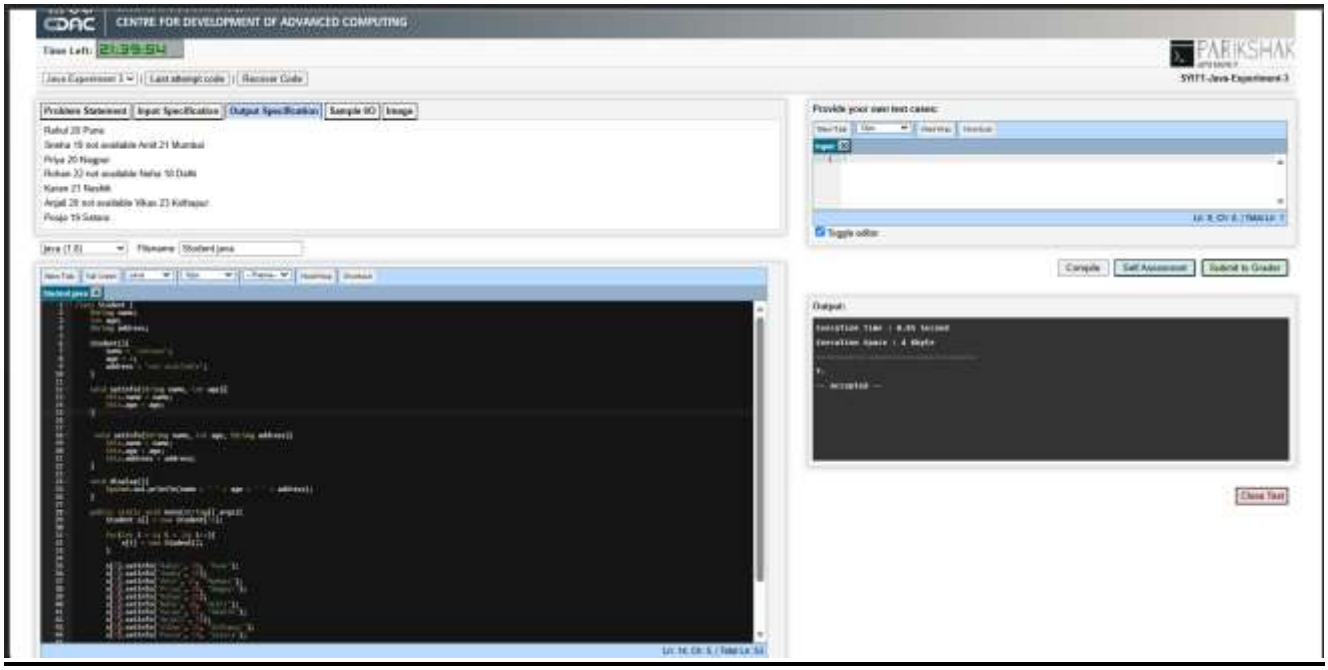
Create a class ‘Student’; with three data members which are name, age and address. The constructor of the class assigns default values name as “unknown”;, age as ‘0’; and address a “not available”. It has two members with the same name ‘setInfo’. First method has two parameters for name and age and assigns the same whereas the second method takes has three parameters which are assigned to name, age and address respectively. Print the name, age and address of 10 students. Hint - Use array of objects.

3.7 Program and Output:

Program –

```
class Student {
   String name;
   int age;
   String address;

   Student() {
      name = "unknown";
      age = 0;
      address = "not available";
   }

   void setInfo(String name, int age) {
      this.name = name;
      this.age = age;
   }

   void setInfo(String name, int age, String address) {
      this.name = name;
      this.age = age;
      this.address = address;
   }
```

```
    void display() {
        System.out.println(name + " " + age + " " + address);
    }

    public static void main(String[] args) {

        Student[] s = new Student[10];

        for (int i = 0; i < 10; i++) {
            s[i] = new Student();
        }

        s[0].setInfo("Rahul", 20, "Pune");
        s[1].setInfo("Sneha", 19);
        s[2].setInfo("Amit", 21, "Mumbai");
        s[3].setInfo("Priya", 20, "Nagpur");
        s[4].setInfo("Rohan", 22);
        s[5].setInfo("Neha", 18, "Delhi");
        s[6].setInfo("Karan", 21, "Nashik");
        s[7].setInfo("Anjali", 20);
        s[8].setInfo("Vikas", 23, "Kolhapur");
        s[9].setInfo("Pooja", 19, "Satara");

        for (int i = 0; i < 10; i++) {
            s[i].display();
        }
    }
}
```



3.8 Conclusion:

This experiment helped in understanding the concept of constructors in Java and their role in object initialization. We learned how a default constructor assigns initial values to object data members and how method overloading can be used with the setInfo() method to initialize objects in different ways. We also learned to create and use an array of objects for storing and displaying details of multiple students. Thus, the experiment improved our understanding of constructors, object creation, method overloading, and arrays of objects in Java.

3.9 Questions:

1. What are Constructors in JAVA?

Ans - A constructor is a special method in Java that is automatically called when an object is created. It is used to initialize the object's data members. A constructor has the same name as the class and does not have any return type.

2. Why are constructors used?

Ans – Constructors are used to:

- Initialize objects with default or user-defined values.
- Reduce the need to assign values separately after object creation.
- Ensure that objects are created in a valid and initialized state.
- Simplify object creation.

3. Can we overload constructors?

Ans – Yes. Constructors can be overloaded in Java by creating multiple constructors with different parameter lists. This allows objects to be initialized in different ways depending on the arguments passed during object creation.

Example:

```
Student() { }
Student(String name, int age) { }
Student(String name, int age, String address) { }
```

4. Can a constructor be private?

Ans - Yes. A constructor can be declared as private. A private constructor prevents objects of the class from being created outside the class. It is commonly used in the **Singleton Design Pattern** and in utility classes where only one object or no object should be created.

……

……